```
"use client";

import { useEffect, useState, useRef } from "react";
import { MoreVertical, FolderOpen, CheckCircle2 } from "lucide-react";
import { useToast, ToastContainer } from "@/components/Toast";
import { ConfirmDialog } from "@/components/ConfirmDialog";
import DescriptionDisplay from "@/components/DescriptionDisplay";
import UrlVideoPopup from "@/components/UrlVideoPopup";

type Option = { id: number; name: string };

type ProgramOption = {
  id: number;
  name: string;
  client?: number;
};

type Assignment = {
  id: number;
  client: Option;
  program: ProgramOption;
  created_at?: string;
};

type Attachment = {
  id: number;
```

```
  file_name: string;
  description: string;
  asset_type: "file" | "url";
  sharepoint_url: string | null;
  external_url: string | null;
  clients: Option[]; // temporary legacy support
  programs: ProgramOption[]; // temporary legacy support
  assignments: Assignment[]; // NEW and important
};

export default function UploadedFilesList({
  refreshKey,
  clientId,
  programId,
}: {
  refreshKey?: number;
  clientId?: number | null;
  programId?: number | null;
}) {
  const API = process.env.NEXT_PUBLIC_API_BASE_URL;

  const [files, setFiles] = useState<Attachment[]>([]);
  const [openMenu, setOpenMenu] = useState<number | null>(null);
  const [search, setSearch] = useState("");
  const [fileIdToDelete, setFileIdToDelete] = useState<number | null>
(null);
  const [videoPopup, setVideoPopup] = useState<{
    title: string;
    videoUrl: string;
```

```
    embedUrl: string | null;
    isDirectVideo: boolean;
  } | null>(null);

  const { toasts, showToast, removeToast } = useToast();

  function notifyAttachmentsUpdated() {
    localStorage.setItem("attachments_updated_at",
Date.now().toString());
    window.dispatchEvent(new Event("attachments-updated"));
  }

  function normalizeAttachment(raw: any): Attachment {
    return {
      id: Number(raw?.id),
      file_name: raw?.file_name || "",
      description: raw?.description || "",
      asset_type: raw?.asset_type || "file",
      sharepoint_url: raw?.sharepoint_url || null,
      external_url: raw?.external_url || null,
      clients: Array.isArray(raw?.clients) ? raw.clients : [],
      programs: Array.isArray(raw?.programs) ? raw.programs : [],
      assignments: Array.isArray(raw?.assignments) ? raw.assignments : [],
    };
  }

  async function fetchFilesREST() {
    try {
      const res = await fetch(`${API}/api/attachments/`, {
```

```
      cache: "no-store",
    });

    const data = await res.json().catch(() => []);

    if (!res.ok) {
      throw new Error(data?.error || "Failed to fetch attachments");
    }

    const normalized = Array.isArray(data)
      ? data.map(normalizeAttachment)
      : [];

    // TEMP DEBUG - remove later if you want
    console.log("ATTACHMENTS API DATA:", normalized);

    setFiles(normalized);
  } catch (err: any) {
    console.error("Failed to fetch attachments:", err);
    showToast(err.message || "Failed to load files", "error");
    setFiles([]);
  }
}

useEffect(() => {
  fetchFilesREST();
}, [refreshKey]);

// IMPORTANT: listen for global updates from other components
```

```
  useEffect(() => {
    function handleAttachmentsUpdated() {
      fetchFilesREST();
    }

    window.addEventListener("attachments-updated",
handleAttachmentsUpdated);
    return () => {
      window.removeEventListener("attachments-updated",
handleAttachmentsUpdated);
    };
  }, []);

  async function assignFileToCurrentSelection(file: Attachment) {
    if (!clientId || !programId) {
      showToast("Please select a Client and Program from the sidebar
first", "error");
      return;
    }

    try {
      const res = await
fetch(`${API}/api/attachments/${file.id}/assign/`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          client_id: Number(clientId),
          program_id: Number(programId),
        }),
```

```
      });

      const data = await res.json().catch(() => ({}));

      if (!res.ok) {
        throw new Error(
          data?.program_id?.[0] ||
            data?.non_field_errors?.[0] ||
            data?.error ||
            "Failed to assign file"
        );
      }

      const updatedAttachment = data?.attachment
        ? normalizeAttachment(data.attachment)
        : null;

      // Update local state immediately for instant UI response
      if (updatedAttachment) {
        setFiles((prev) =>
          prev.map((f) => (f.id === file.id ? updatedAttachment : f))
        );
      }

      setOpenMenu(null);

      showToast(
        data?.created === false
          ? "File already assigned to selected client/program"
```

```
        : "File assigned successfully",
      "success"
    );

    notifyAttachmentsUpdated();

    // SAFEST: force re-fetch so assigned section is always correct
    await fetchFilesREST();
  } catch (err: any) {
    showToast(err.message || "Assignment failed", "error");
  }
}

async function unassignFile(file: Attachment) {
  if (!clientId || !programId) {
    showToast("Please select a Client and Program first", "error");
    return;
  }

  try {
    const res = await
fetch(`${API}/api/attachments/${file.id}/unassign/`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        client_id: Number(clientId),
        program_id: Number(programId),
      }),
    });
```

```
const data = await res.json().catch(() => ({}));

if (!res.ok) {
  throw new Error(
    data?.program_id?.[0] ||
      data?.non_field_errors?.[0] ||
      data?.error ||
      "Failed to unassign file"
  );
}

const updatedAttachment = data?.attachment
  ? normalizeAttachment(data.attachment)
  : null;

// Update local state immediately for instant UI response
if (updatedAttachment) {
  setFiles((prev) =>
    prev.map((f) => (f.id === file.id ? updatedAttachment : f))
  );
}

setOpenMenu(null);

showToast(
  data?.deleted
    ? "File unassigned successfully"
    : "File was already unassigned",
```

```
      "success"
    );

    notifyAttachmentsUpdated();

    // SAFEST: force re-fetch so assigned section is always correct
    await fetchFilesREST();
  } catch (err: any) {
    showToast(err.message || "Unassign failed", "error");
  }
}

function requestDeleteFile(fileId: number) {
  setOpenMenu(null);
  setFileIdToDelete(fileId);
}

async function confirmDeleteFile() {
  if (fileIdToDelete == null) return;

  const fileId = fileIdToDelete;
  setFileIdToDelete(null);

  try {
    const res = await fetch(`${API}/api/attachments/${fileId}/`, {
      method: "DELETE",
    });

    const data = await res.json().catch(() => ({}));
```

```
    if (!res.ok) {
      throw new Error(data?.error || "Failed to delete file");
    }

    setFiles((prev) => prev.filter((f) => f.id !== fileId));
    showToast("File deleted successfully", "success");
    notifyAttachmentsUpdated();

    // optional safety refetch
    await fetchFilesREST();
  } catch (err: any) {
    showToast(err.message || "Delete failed", "error");
  }
}

function isDirectVideoUrl(url: string) {
  const lower = url.toLowerCase();
  return (
    lower.endsWith(".mp4") ||
    lower.endsWith(".webm") ||
    lower.endsWith(".ogg") ||
    lower.endsWith(".mov") ||
    lower.endsWith(".m4v")
  );
}

function getEmbedUrl(url: string): string | null {
  try {
```

```
    const parsed = new URL(url);
    const host = parsed.hostname.toLowerCase();

    // YouTube short link: youtu.be/VIDEO_ID
    if (host.includes("youtu.be")) {
      const videoId = parsed.pathname.replace("/", "");
      return videoId ? `https://www.youtube.com/embed/${videoId}` :
null;
    }

    // YouTube normal link: youtube.com/watch?v=VIDEO_ID
    if (host.includes("youtube.com")) {
      const videoId = parsed.searchParams.get("v");
      if (videoId) return `https://www.youtube.com/embed/${videoId}`;

      // already embed url
      if (parsed.pathname.includes("/embed/")) return url;
    }

    // Vimeo
    if (host.includes("vimeo.com")) {
      const pathParts = parsed.pathname.split("/").filter(Boolean);
      const videoId = pathParts[pathParts.length - 1];
      return videoId ? `https://player.vimeo.com/video/${videoId}` :
null;
    }

    // Loom
    if (host.includes("loom.com")) {
```

```
      const pathParts = parsed.pathname.split("/").filter(Boolean);

      const shareIndex = pathParts.findIndex((part) => part ===
"share");
      if (shareIndex !== -1 && pathParts[shareIndex + 1]) {
        return https://www.loom.com/embed/${pathParts[shareIndex +
1]};
      }

      const embedIndex = pathParts.findIndex((part) => part ===
"embed");
      if (embedIndex !== -1 && pathParts[embedIndex + 1]) {
        return url;
      }
    }

    return null;
  } catch {
    return null;
  }
}

function isPlayableVideoUrl(url: string) {
  return isDirectVideoUrl(url) || !!getEmbedUrl(url);
}

function openUrlVideo(file: Attachment) {
  if (!file.external_url) {
    showToast("URL not available", "error");
```

```
    return;
  }

  const directVideo = isDirectVideoUrl(file.external_url);
  const embedUrl = getEmbedUrl(file.external_url);

  if (!directVideo && !embedUrl) {
    window.open(file.external_url, "_blank", "noopener,noreferrer");
    return;
  }

  setVideoPopup({
    title: file.file_name,
    videoUrl: file.external_url,
    embedUrl,
    isDirectVideo: directVideo,
  });
}

const filteredFiles = files.filter((f) =>
  f.file_name.toLowerCase().includes(search.toLowerCase())
);

// Right side:
// Files assigned to CURRENT selected sidebar client + program
const assigned = filteredFiles.filter((f) => {
  if (!clientId || !programId) return false;

  return (f.assignments || []).some(
```

```
    (a) =>
      Number(a.client?.id) === Number(clientId) &&
      Number(a.program?.id) === Number(programId)
  );
});

// Left side:
// Files NOT assigned to CURRENT selected sidebar client + program
const available = filteredFiles.filter((f) => {
  if (!clientId || !programId) {
    // show only files with no exact assignments
    return (f.assignments || []).length === 0;
  }

  const isAssignedToCurrentSelection = (f.assignments || []).some(
    (a) =>
      Number(a.client?.id) === Number(clientId) &&
      Number(a.program?.id) === Number(programId)
  );

  return !isAssignedToCurrentSelection;
});

function FileCard({
  file,
  isAvailableSection,
}: {
  file: Attachment;
  isAvailableSection: boolean;
```

```
  }) {
    const cardRef = useRef<HTMLDivElement>(null);

    const isTrulyUnassigned = (file.assignments || []).length === 0;

    useEffect(() => {
      function handleClickOutside(event: MouseEvent) {
        if (
          openMenu === file.id &&
          cardRef.current &&
          !cardRef.current.contains(event.target as Node)
        ) {
          setOpenMenu(null);
        }
      }

      document.addEventListener("mousedown", handleClickOutside);
      return () => {
        document.removeEventListener("mousedown",
handleClickOutside);
      };
    }, [openMenu, file.id]);

    return (
      <div
        ref={cardRef}
        className="relative border rounded-xl p-4 bg-white dark:bg-
slate-900 border-slate-200 dark:border-slate-700 shadow-sm
hover:shadow-md transition"
```

```
      >
        <button
          className="absolute top-3 right-3 text-slate-500 dark:text-
slate-400"
          onClick={() => setOpenMenu(openMenu === file.id ? null :
file.id)}
        >
          <MoreVertical size={18} />
        </button>

        <p className="font-semibold pr-8 break-words">{file.file_name}
</p>

        {file.description && (
          <DescriptionDisplay
            html={file.description}
            className="text-xs text-slate-500 dark:text-slate-400"
          />
        )}

        <p className="text-xs">
          Clients: {file.clients.map((c) => c.name).join(", ") || "None"}
        </p>

        <p className="text-xs">
          Programs: {file.programs.map((p) => p.name).join(", ") || "None"}
        </p>

        {file.asset_type === "file" && file.sharepoint_url ? (
```

```
          <a
            href={file.sharepoint_url}
            target="_blank"
            rel="noopener noreferrer"
            className="text-xs text-emerald-600 hover:text-emerald-700
dark:text-emerald-400 dark:hover:text-emerald-300"
          >
            View File
          </a>
        ) : file.asset_type === "url" && file.external_url ? (
          isPlayableVideoUrl(file.external_url) ? (
            <button
              onClick={() => openUrlVideo(file)}
              className="mt-2 inline-flex items-center gap-2 px-3 py-1.5
text-xs rounded-lg bg-emerald-600 text-white hover:bg-emerald-700
transition"
            >
              ▶️ Play Video
            </button>
          ) : (
            <a
              href={file.external_url}
              target="_blank"
              rel="noopener noreferrer"
              className="text-xs text-emerald-600 hover:text-emerald-700
dark:text-emerald-400 dark:hover:text-emerald-300"
            >
              Open URL
            </a>
```

```
          )
        ) : (
          <span className="text-xs text-slate-400 dark:text-slate-500">
            No link available
          </span>
        )}

        {openMenu === file.id && (
          <div
            className="
              absolute right-3 top-10
              w-[180px]
              bg-white dark:bg-slate-900
              border border-slate-200 dark:border-slate-700
              p-3 rounded-lg shadow-lg
              space-y-2 z-50
            "
          >
            {isAvailableSection ? (
              <>
                <button
                  onClick={() => assignFileToCurrentSelection(file)}
                  className="w-full px-3 py-2 text-sm rounded bg-emerald-
600 text-white hover:bg-emerald-700"
                >
                  Assign
                </button>

                {/* Delete only if file is truly global unassigned */}
```

```
                {isTrulyUnassigned && (
                  <button
                    onClick={() => requestDeleteFile(file.id)}
                    className="w-full px-3 py-2 text-sm rounded bg-red-600
text-white hover:bg-red-700"
                  >
                    Delete
                  </button>
                )}
              </>
            ) : (
              <button
                onClick={() => unassignFile(file)}
                className="w-full px-3 py-2 text-sm rounded bg-amber-500
text-white hover:bg-amber-600"
              >
                Unassign
              </button>
            )}
          </div>
        )}
      </div>
    );
  }

  return (
    <div className="space-y-6">
      <div className="mb-4 flex justify-center">
        <div className="relative w-full max-w-md">
```

```
          <svg
            className="absolute left-3 top-1/2 -translate-y-1/2 w-4 h-4
text-slate-400 dark:text-slate-500"
            fill="none"
            stroke="currentColor"
            viewBox="0 0 24 24"
          >
            <path
              strokeLinecap="round"
              strokeLinejoin="round"
              strokeWidth={2}
              d="M21 21l-6-6m2-5a7 7 0 11-14 0 7 7 0 0114 0z"
            />
          </svg>

          <input
            type="text"
            placeholder="Search files..."
            value={search}
            onChange={(e) => setSearch(e.target.value)}
            className="w-full pl-10 pr-4 py-2 border border-slate-200
dark:border-slate-700 rounded-lg bg-white dark:bg-slate-900 text-
slate-900 dark:text-slate-100 focus:outline-none focus:ring-2
focus:ring-emerald-500"
          />
        </div>
      </div>

      <ConfirmDialog
```

```
        open={fileIdToDelete != null}
        title="Delete file?"
        description="This action cannot be undone."
        confirmLabel="Delete"
        cancelLabel="Cancel"
        variant="danger"
        onConfirm={confirmDeleteFile}
        onCancel={() => setFileIdToDelete(null)}
      />

      <ToastContainer toasts={toasts} onClose={removeToast} />

      <UrlVideoPopup
        open={videoPopup != null}
        title={videoPopup?.title || "Video Player"}
        videoUrl={videoPopup?.videoUrl || null}
        embedUrl={videoPopup?.embedUrl || null}
        isDirectVideo={videoPopup?.isDirectVideo || false}
        onClose={() => setVideoPopup(null)}
      />

      <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
        {/* Available Files */}
        <div className="bg-white dark:bg-slate-900 p-4 rounded-xl
border border-slate-200 dark:border-slate-700">
          <h3 className="mb-3 text-slate-900 dark:text-slate-100 inline-
flex items-center gap-2">
            <FolderOpen className="h-4 w-4 text-slate-500 dark:text-
slate-400" aria-hidden />
```

```
          Available Files
        </h3>

        <div className="space-y-3 h-[500px] overflow-y-auto">
          {available.length > 0 ? (
            available.map((file) => (
              <FileCard
                key={file.id}
                file={file}
                isAvailableSection={true}
              />
            ))
          ) : (
            <div className="text-center py-8 text-slate-500 dark:text-
slate-400">
              No available files match your search
            </div>
          )}
        </div>
      </div>

      {/* Assigned Files */}
      <div className="bg-white dark:bg-slate-900 p-4 rounded-xl
border border-slate-200 dark:border-slate-700">
        <h3 className="mb-3 text-slate-900 dark:text-slate-100 inline-
flex items-center gap-2">
          <CheckCircle2 className="h-4 w-4 text-emerald-600 dark:text-
emerald-400" aria-hidden />
          Assigned Files
```

```
          </h3>

          <div className="space-y-3 h-[500px] overflow-y-auto">
            {assigned.length > 0 ? (
              assigned.map((file) => (
                <FileCard
                  key={file.id}
                  file={file}
                  isAvailableSection={false}
                />
              ))
            ) : (
              <div className="text-center py-8 text-slate-500 dark:text-
slate-400">
                {!clientId || !programId
                  ? "Select a client and program from the sidebar to view
assigned files"
                  : "No assigned files match your search"}
              </div>
            )}
          </div>
        </div>
      </div>
    </div>
  );
}
```

this is my uploadedfileslist.tsx
in assigned files section there is unassign button i want to add one more button "UPDATE" when we click on update it means update file

like keep its name and assigned clients and programs as it it dont change anything just we have to update file content or url whatever conten is there is it possible here tell me how to do it proparlly step by step in detail

```
"use client";

import { useEffect, useState } from "react";
import { Loader2 } from "lucide-react";
import { getCSRFToken } from "@/lib/csrf";
import RichTextEditor from "@/components/RichTextEditor";

/** 1GB limit */
const MAX_FILE_SIZE = 1024 * 1024 * 1024;

/** Allowed MIME prefixes & types */
const ALLOWED_TYPES = [
  "image/",
  "application/pdf",
  "application/msword",
  "application/vnd.ms-excel",
  "application/vnd.ms-powerpoint",
  "application/vnd.openxmlformats-officedocument",
  "text/plain",
  "video/",
  "audio/",
];

export default function AttachmentUploader({
  onUploadSuccess,
```

```
}: {
  onUploadSuccess?: () => void;
}) {
  const [file, setFile] = useState<File | null>(null);
  const [name, setName] = useState("");
  const [description, setDescription] = useState("");
  const [url, setUrl] = useState("");
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [success, setSuccess] = useState<string | null>(null);

  useEffect(() => {
    if (file && !name) {
      setName(file.name);
    }
  }, [file, name]);

  function validateFile(file: File) {
    if (file.size > MAX_FILE_SIZE) {
      return "File size exceeds 1GB limit";
    }

    const isAllowed = ALLOWED_TYPES.some((type) =>
      file.type.startsWith(type)
    );

    if (!isAllowed) {
      return "Unsupported file type";
    }
```

```
  return null;
}

async function handleUpload() {
  setError(null);
  setSuccess(null);

  if (!name.trim()) {
    setError("Asset name is required");
    return;
  }

  if (!file && !url.trim()) {
    setError("Please upload a file or provide a URL");
    return;
  }

  if (file) {
    const validationError = validateFile(file);
    if (validationError) {
      setError(validationError);
      return;
    }
  }

  setLoading(true);

  const fd = new FormData();
```

```
if (file) {
  fd.append("file", file);
}

if (url.trim()) {
  fd.append("external_url", url.trim());
}

fd.append("name", name);
fd.append("description", description);

try {
  const headers: Record<string, string> = {};

  const token = getCSRFToken();
  if (token) {
    headers["X-CSRFToken"] = token;
  }

  const res = await fetch(

`${process.env.NEXT_PUBLIC_API_BASE_URL}/api/attachments/upload/`,
    {
      method: "POST",
      body: fd,
      credentials: "include",
      headers,
    }
```

```
      );

      if (!res.ok) {
        const data = await res.json().catch(() => null);
        throw new Error(
          data?.error ||
          data?.detail ||
          "Upload failed"
        );
      }

      setFile(null);
      setName("");
      setDescription("");
      setUrl("");

      setSuccess("Asset uploaded successfully");

      onUploadSuccess?.();
    } catch (err: any) {
      setError(err.message || "Upload error");
    } finally {
      setLoading(false);
    }
  }

  return (
    <div className="bg-white dark:bg-slate-900 border border-slate-
200 dark:border-slate-700 rounded-xl shadow-sm p-6 space-y-4 text-
```

```
slate-900 dark:text-slate-100">
      <h2 className="text-lg font-semibold">Upload Asset</h2>

      {/* FILE INPUT */}
      <div>
        <label className="text-sm block mb-1 text-slate-700 dark:text-
slate-300">
          Select File (Optional)
        </label>
        <input
          type="file"
          onChange={(e) => setFile(e.target.files?.[0] || null)}
          className="w-full text-sm text-slate-700 dark:text-slate-300
file:mr-3 file:rounded-md file:border-0 file:bg-slate-100 dark:file:bg-
slate-800 file:px-3 file:py-1.5 file:text-slate-700 dark:file:text-slate-200"
        />
      </div>

      {/* NAME */}
      <div>
        <label className="text-sm block mb-1 text-slate-700 dark:text-
slate-300">
          Asset Name *
        </label>
        <input
          className="w-full border border-slate-300 dark:border-slate-
600 bg-white dark:bg-slate-800 text-slate-900 dark:text-slate-100
placeholder:text-slate-400 dark:placeholder:text-slate-500 rounded-lg
px-3 py-2 text-sm"
```

```
          placeholder="Asset name"
          value={name}
          onChange={(e) => setName(e.target.value)}
        />
      </div>

      {/* DESCRIPTION */}
      <div>
        <label className="text-sm block mb-1 text-slate-700 dark:text-
slate-300">
          Description
        </label>
        <RichTextEditor
          value={description}
          onChange={setDescription}
          rows={3}
          className="border-slate-300 dark:border-slate-600"
        />
      </div>

      {/* URL FIELD */}
      <div>
        <label className="text-sm block mb-1 text-slate-700 dark:text-
slate-300">
          External URL (Optional)
        </label>
        <input
          type="url"
          className="w-full border border-slate-300 dark:border-slate-
```

```
600 bg-white dark:bg-slate-800 text-slate-900 dark:text-slate-100
placeholder:text-slate-400 dark:placeholder:text-slate-500 rounded-lg
px-3 py-2 text-sm"
          placeholder="https://example.com"
          value={url}
          onChange={(e) => setUrl(e.target.value)}
        />
      </div>

      {/* BUTTON */}
      <button
        onClick={handleUpload}
        disabled={loading}
        className="px-6 py-2 rounded-lg bg-emerald-600 hover:bg-
emerald-700 text-white flex items-center gap-2 disabled:opacity-60"
      >
        {loading && (
          <Loader2 className="h-4 w-4 animate-spin" />
        )}
        Upload
      </button>

      {error && (
        <p className="text-sm text-red-600 dark:text-red-400">
          {error}
        </p>
      )}

      {success && (
```

```
        <p className="text-sm text-emerald-600 dark:text-emerald-
400">
          {success}
        </p>
      )}
    </div>
  );
}this is attachmentuploader.tsx if required
now you can proceed
```

Yes — **this is absolutely possible** ✅

And your requirement is very clear:

> In **Assigned Files** section, add **UPDATE** button
> When clicked:
> - **"keep same asset name"**
> - **"keep same description (unless you want to edit later)"**
> - **"keep same assigned clients/programs / assignments"**
> - **"only replace:"**
> - **"uploaded file content OR"**
> - **"external URL"**
> - **"do not change assignment mapping"**

That is the **correct approach**.

Best way to implement it

You should do this in **2 parts**:

1. **Frontend**
   - Add **Update** button in `UploadedFilesList.tsx`
   - Open an **Update Modal/Dialog**
   - In that modal:
     - show current name (readonly or editable if you want)
     - show current description (readonly or editable if you want)
     - allow choosing:
       - new file **or**
       - new URL
   - Submit to backend
2. **Backend (important)**
   - Create a new API endpoint like:

Python Run

or

Python Run

This endpoint should:

- replace file or external URL
- update `asset_type`
- update `sharepoint_url` / `external_url`

- **NOT touch assignments**
- **NOT delete/recreate attachment row**
- keep same `id`

👉 **This is the safest and best architecture.**

VERY IMPORTANT DESIGN DECISION

❌ Do NOT delete old attachment and create new one

Because then:

- `id` changes
- assignments can break
- foreign keys may break
- UI state becomes messy

✅ Update the existing attachment record in place

This keeps:

- same `attachment.id`
- same assignments
- same client/program relations
- same list placement

Step-by-step proper implementation

PART 1) Add Update button in `UploadedFilesList.tsx`

Inside your `FileCard` menu for **assigned section**, currently you have only:

```
<button
  onClick={() => unassignFile(file)}
  className="w-full px-3 py-2 text-sm rounded bg-amber-500 text-white hover:bg-amber-600"
>
  Unassign
</button>
```

You will change it to:

```
<>
  <button
    onClick={() => openUpdateModal(file)}
    className="w-full px-3 py-2 text-sm rounded bg-blue-600 text-white hover:bg-blue-700"
  >
    Update
  </button>

  <button
    onClick={() => unassignFile(file)}
    className="w-full px-3 py-2 text-sm rounded bg-amber-500 text-white hover:bg-amber-600"
  >
    Unassign
  </button>
</>
```

PART 2) Add update modal state in `UploadedFilesList.tsx`

Add these states near top:

```
const [fileToUpdate, setFileToUpdate] = useState<Attachment | null>(null);
const [updateFile, setUpdateFile] = useState<File | null>(null);
const [updateUrl, setUpdateUrl] = useState("");
```

TypeScript

PART 3) Add helper to open update modal

Add this function:

```
function openUpdateModal(file: Attachment) {
  setOpenMenu(null);
  setFileToUpdate(file);
  setUpdateFile(null);
  setUpdateUrl(file.external_url || "");
}
```

PART 4) Add file validation helpers (same as uploader)

Since your uploader already has validation, reuse similar logic here.

Add near helper functions:

```
const MAX_FILE_SIZE = 1024 * 1024 * 1024;

const ALLOWED_TYPES = [
  "image/",
  "application/pdf",
  "application/msword",
  "application/vnd.ms-excel",
  "application/vnd.ms-powerpoint",
  "application/vnd.openxmlformats-officedocument",
  "text/plain",
  "video/",
  "audio/",
];

function validateFile(file: File) {
  if (file.size > MAX_FILE_SIZE) {
    return "File size exceeds 1GB limit";
  }

  const isAllowed = ALLOWED_TYPES.some((type) =>
    file.type.startsWith(type)
  );

  if (!isAllowed) {
    return "Unsupported file type";
  }

  return null;
}
```

PART 5) Add update submit function

This is the core frontend logic.

Add this function:

```
async function confirmUpdateFile() {
  if (!fileToUpdate) return;

  if (!updateFile && !updateUrl.trim()) {
    showToast("Please select a new file or provide a new URL", "error");
    return;
  }

  if (updateFile) {
    const validationError = validateFile(updateFile);
    if (validationError) {
      showToast(validationError, "error");
      return;
    }
  }

  setUpdateLoading(true);

  try {
    const fd = new FormData();

    if (updateFile) {
      fd.append("file", updateFile);
    }

    if (updateUrl.trim()) {
      fd.append("external_url", updateUrl.trim());
```

```
    }

    // Optional: if backend expects existing values too
    fd.append("name", fileToUpdate.file_name);
    fd.append("description", fileToUpdate.description || "");

    const res = await fetch(
      `${API}/api/attachments/${fileToUpdate.id}/replace/`,
      {
        method: "POST",
        body: fd,
        credentials: "include",
      }
    );

    const data = await res.json().catch(() => ({}));

    if (!res.ok) {
      throw new Error(data?.error || "Failed to update file");
    }

    const updatedAttachment = data?.attachment
      ? normalizeAttachment(data.attachment)
      : null;

    if (updatedAttachment) {
      setFiles((prev) =>
        prev.map((f) => (f.id === updatedAttachment.id ? updatedAttachment : f))
      );
    }

    setFileToUpdate(null);
    setUpdateFile(null);
    setUpdateUrl("");
```

```
      showToast("File updated successfully", "success");

      notifyAttachmentsUpdated();
      await fetchFilesREST();
    } catch (err: any) {
      showToast(err.message || "Update failed", "error");
    } finally {
      setUpdateLoading(false);
    }
  }
```

PART 6) Add the Update modal UI

Since you already use `ConfirmDialog`, but this update flow needs:

- file input
- URL input
- maybe readonly name/description

So **best** is create a new reusable modal component, e.g.:

- `UpdateAttachmentDialog.tsx`

Create `components/UpdateAttachmentDialog.tsx`

Use this:

```
"use client";
```

```
import { Loader2 } from "lucide-react";

export default function UpdateAttachmentDialog({
  open,
  attachmentName,
  description,
  currentUrl,
  updateUrl,
  setUpdateUrl,
  setUpdateFile,
  onClose,
  onConfirm,
  loading,
}: {
  open: boolean;
  attachmentName: string;
  description: string;
  currentUrl: string | null;
  updateUrl: string;
  setUpdateUrl: (value: string) => void;
  setUpdateFile: (file: File | null) => void;
  onClose: () => void;
  onConfirm: () => void;
  loading: boolean;
}) {
  if (!open) return null;

  return (
    <div className="fixed inset-0 z-[100] flex items-center justify-center bg-black/50 px-4">
      <div className="w-full max-w-lg rounded-2xl bg-white dark:bg-slate-900 border border-slate-
        <h2 className="text-lg font-semibold text-slate-900 dark:text-slate-100">
          Update File Content
        </h2>

        <div>
```

```
            <label className="text-sm block mb-1 text-slate-700 dark:text-slate-300">
              Asset Name
            </label>
            <input
              value={attachmentName}
              readOnly
              className="w-full rounded-lg border border-slate-300 dark:border-slate-600 bg-slate-1
            />
          </div>

          <div>
            <label className="text-sm block mb-1 text-slate-700 dark:text-slate-300">
              Description
            </label>
            <textarea
              value={description}
              readOnly
              rows={3}
              className="w-full rounded-lg border border-slate-300 dark:border-slate-600 bg-slate-1
            />
          </div>

          <div>
            <label className="text-sm block mb-1 text-slate-700 dark:text-slate-300">
              Select New File (Optional)
            </label>
            <input
              type="file"
              onChange={(e) => setUpdateFile(e.target.files?.[0] || null)}
              className="w-full text-sm text-slate-700 dark:text-slate-300 file:mr-3 file:rounded-m
            />
          </div>

          <div>
            <label className="text-sm block mb-1 text-slate-700 dark:text-slate-300">
```

```
            New External URL (Optional)
          </label>
          <input
            type="url"
            value={updateUrl}
            onChange={(e) => setUpdateUrl(e.target.value)}
            placeholder="https://example.com"
            className="w-full rounded-lg border border-slate-300 dark:border-slate-600 bg-white d
          />
          {currentUrl && (
            <p className="mt-1 text-xs text-slate-500 dark:text-slate-400 break-all">
              Current URL: {currentUrl}
            </p>
          )}
        </div>

        <p className="text-xs text-slate-500 dark:text-slate-400">
          Name and assignments will remain unchanged. Only file content or URL will be replaced.
        </p>

        <div className="flex justify-end gap-3 pt-2">
          <button
            onClick={onClose}
            disabled={loading}
            className="px-4 py-2 rounded-lg border border-slate-300 dark:border-slate-600 text-sm
          >
            Cancel
          </button>

          <button
            onClick={onConfirm}
            disabled={loading}
            className="px-4 py-2 rounded-lg bg-blue-600 hover:bg-blue-700 text-white text-sm flex
          >
            {loading && <Loader2 className="h-4 w-4 animate-spin" />}
```

```
            Update
          </button>
        </div>
      </div>
    </div>
  );
}
```

PART 7) Use the modal inside `UploadedFilesList.tsx`

Add import:

TypeScript

Then in JSX near `ConfirmDialog` / `ToastContainer` / `UrlVideoPopup`, add:

```
<UpdateAttachmentDialog
  open={fileToUpdate != null}
  attachmentName={fileToUpdate?.file_name || ""}
  description={fileToUpdate?.description || ""}
  currentUrl={fileToUpdate?.external_url || null}
  updateUrl={updateUrl}
  setUpdateUrl={setUpdateUrl}
  setUpdateFile={setUpdateFile}
  onClose={() => {
    setFileToUpdate(null);
    setUpdateFile(null);
```

```
      setUpdateUrl("");
    }}
    onConfirm={confirmUpdateFile}
    loading={updateLoading}
  />
```

PART 8) Backend — MOST IMPORTANT PART

Your frontend is only half.

You need backend endpoint:

Python Run

This endpoint should:

- fetch existing `Attachment`
- if new file uploaded:
  - upload to SharePoint (or wherever your existing upload logic goes)
  - set:
    - `asset_type = "file"`
    - `sharepoint_url = new_uploaded_file_url`
    - `external_url = None`
- if new URL provided:
  - set:
    - `asset_type = "url"`
    - `external_url = new_url`

    - `sharepoint_url = None`
- keep:
    - `id`
    - `name`
    - `description`
    - `clients`
    - `programs`
    - `assignments`

Example Django backend logic (recommended)

Assuming you have DRF or normal Django view.

URL

Python Run

View example

```python
@api_view(["POST"])
@permission_classes([IsAuthenticated])
def replace_attachment(request, pk):
    try:
```

```
        attachment = Attachment.objects.get(pk=pk)
    except Attachment.DoesNotExist:
        return Response({"error": "Attachment not found"}, status=404)

    uploaded_file = request.FILES.get("file")
    external_url = request.data.get("external_url", "").strip()

    if not uploaded_file and not external_url:
        return Response(
            {"error": "Please provide a new file or external URL"},
            status=400,
        )

    try:
        # CASE 1: replace with uploaded file
        if uploaded_file:
            # Reuse your existing SharePoint upload function here
            # Example:
            # sharepoint_url = upload_file_to_sharepoint(uploaded_file)

            sharepoint_url = your_existing_sharepoint_upload_function(uploaded_file)

            attachment.asset_type = "file"
            attachment.sharepoint_url = sharepoint_url
            attachment.external_url = None

        # CASE 2: replace with URL
        elif external_url:
            attachment.asset_type = "url"
            attachment.external_url = external_url
            attachment.sharepoint_url = None

        # Keep existing name/description unless you want optional editing
        attachment.save()
```

```
        serializer = AttachmentSerializer(attachment, context={"request": request})
        return Response(
            {
                "message": "Attachment updated successfully",
                "attachment": serializer.data,
            },
            status=200,
        )

    except Exception as e:
        return Response({"error": str(e)}, status=500)
```

PART 9) If your model uses `name` but frontend uses `file_name`

I noticed:

In uploader you send:

TypeScript

But in frontend display you use:

TypeScript

So your serializer is probably mapping model `name` → API `file_name`.

That's fine 👍

Just make sure backend serializer returns same shape:

```
{
  "id": 1,
  "file_name": "My Asset.pdf",
  "description": "...",
  "asset_type": "file",
  "sharepoint_url": "...",
  "external_url": null,
  "clients": [...],
  "programs": [...],
  "assignments": [...]
}
```

PART 10) Small UX improvement (recommended)

In the **Update** modal:

Best rule:

Allow **either**

- new file
  OR
- new URL

but not both at same time.

So in `confirmUpdateFile()` add:

```
if (updateFile && updateUrl.trim()) {
  showToast("Please provide either a new file or a new URL, not both", "error");
  return;
`
```

TypeScript

This avoids ambiguity.

PART 11) Even better UX (recommended)

If the original attachment is `asset_type === "file"`:

- default show file replacement section first

If original is `asset_type === "url"`:

- prefill URL field with existing URL

You're already partly doing that with:

TypeScript

That's good.

PART 12) What happens after update?

After successful update:

- same file card remains

- same assignment remains
- if it was assigned in current client/program, it stays in **Assigned Files**
- only link/video/file content changes

That matches exactly what you want.

Recommended final behavior summary

For assigned file card menu:

- **Update**
- **Unassign**

For available file card menu:

- **Assign**
- **Delete** (only if globally unassigned)

This is a clean and correct UX.

IMPORTANT backend caution for SharePoint files

Since you likely upload actual files to SharePoint:

When replacing a file:

- simplest safe approach:
  - upload **new file**
  - save new SharePoint URL

    - leave old file in SharePoint (optional cleanup later)
- advanced approach:
    - delete old SharePoint file after successful new upload

👉 **Recommended initially:**
Do NOT delete old SharePoint file immediately
Just upload new one and replace URL in DB.

Why?

- safer
- avoids accidental data loss
- easier debugging

Later you can add cleanup job if needed.

Exact minimal frontend changes summary

In `UploadedFilesList.tsx` you need:

Add state

```
const [fileToUpdate, setFileToUpdate] = useState<Attachment | null>(null);
const [updateFile, setUpdateFile] = useState<File | null>(null);
const [updateUrl, setUpdateUrl] = useState("");
```

Add functions

- `openUpdateModal(file)`
- `confirmUpdateFile()`
- `validateFile(file)`

Change assigned menu

Replace:

TypeScript

With:

```
<>
  <button onClick={() => openUpdateModal(file)}>Update</button>
  <button onClick={() => unassignFile(file)}>Unassign</button>
```

TypeScript

Add modal component

- `UpdateAttachmentDialog.tsx`

Add backend endpoint

- `POST /api/attachments/<id>/replace/`

My strong recommendation (best production approach)

Create separate update endpoint

Instead of reusing upload endpoint.

Why?

Because upload endpoint is for:

- creating new attachment

Update endpoint is for:

- replacing content of existing attachment

This keeps logic clean and prevents bugs.

Use:

Python Run

This is the best design for your case.

If you want, I can do the next step for you right now

I can now give you **ready-to-paste full code** for:

1. **Updated** `UploadedFilesList.tsx` (complete file with Update button + modal integration)
2. **New** `UpdateAttachmentDialog.tsx` (full component)
3. **Django backend** `replace_attachment` **view + URL + serializer notes**

If you want, reply:

“give me full ready-to-paste frontend + backend code”

and I’ll write it **properly in your project style**, so you can paste it directly with minimal edits.

give me all above steps step by step proparlly first give me all ui steps once it is done then we will do backend

Perfect 👍
That is the **best way** to do it.

We will do this in **2 phases** exactly like you said:

PHASE 1: UI / FRONTEND ONLY

We will complete **all frontend changes first** in a proper clean way.

After that, you test UI flow.

Then in next message we will do:

PHASE 2: BACKEND

- Django API endpoint
- URL route
- replace file/url logic
- keep assignments unchanged

WHAT WE WILL BUILD IN UI (FRONTEND)

Inside **Assigned Files** card menu:

- **Update** button
- **Unassign** button

When user clicks **Update**:

- open a popup/modal
- show:
  - current asset name (**readonly**)
  - current description (**readonly**)
  - new file input (**optional**)
  - new external URL input (**optional**)
- user can provide:
  - new file **OR**
  - new URL
- click **Update**
- frontend will call backend endpoint later (we'll wire now, backend later)

⚠️ For now backend is not created, so UI can be completed and the API call will fail until we add backend in phase 2 — that is expected.

FRONTEND FILES WE WILL TOUCH

You currently have:

1. `UploadedFilesList.tsx` ✅ (main file list)
2. `AttachmentUploader.tsx` ✅ (not required right now)

3. We will create **new component**:
   - `UpdateAttachmentDialog.tsx`

FRONTEND IMPLEMENTATION PLAN (STEP BY STEP)

We will do **exactly 7 UI steps**:

1. **Create new modal component** `UpdateAttachmentDialog.tsx`
2. **Import it** in `UploadedFilesList.tsx`
3. **Add update state variables**
4. **Add file validation helpers**
5. **Add** `openUpdateModal()` **function**
6. **Add** `confirmUpdateFile()` **function**
7. **Add Update button in Assigned Files menu + render modal in JSX**

STEP 1) Create new component: `UpdateAttachmentDialog.tsx`

📁 Create file:

```bash
```

(If your project uses `components/` directly under root alias `@/components`, place it there accordingly.)

Paste this full code inside `UpdateAttachmentDialog.tsx`

```
"use client";

import { Loader2 } from "lucide-react";

export default function UpdateAttachmentDialog({
  open,
  attachmentName,
  description,
  currentUrl,
  updateUrl,
  setUpdateUrl,
  setUpdateFile,
  onClose,
  onConfirm,
  loading,
}: {
  open: boolean;
  attachmentName: string;
  description: string;
  currentUrl: string | null;
  updateUrl: string;
  setUpdateUrl: (value: string) => void;
  setUpdateFile: (file: File | null) => void;
  onClose: () => void;
  onConfirm: () => void;
  loading: boolean;
}) {
  if (!open) return null;

  return (
    <div className="fixed inset-0 z-[100] flex items-center justify-center bg-black/50 px-4">
```

```
        <div className="w-full max-w-lg rounded-2xl bg-white dark:bg-slate-900 border border-slate-
          <h2 className="text-lg font-semibold text-slate-900 dark:text-slate-100">
            Update File Content
          </h2>

          {/* Asset Name (readonly) */}
          <div>
            <label className="text-sm block mb-1 text-slate-700 dark:text-slate-300">
              Asset Name
            </label>
            <input
              value={attachmentName}
              readOnly
              className="w-full rounded-lg border border-slate-300 dark:border-slate-600 bg-slate-1
            />
          </div>

          {/* Description (readonly) */}
          <div>
            <label className="text-sm block mb-1 text-slate-700 dark:text-slate-300">
              Description
            </label>
            <textarea
              value={description}
              readOnly
              rows={3}
              className="w-full rounded-lg border border-slate-300 dark:border-slate-600 bg-slate-1
            />
          </div>

          {/* File input */}
          <div>
            <label className="text-sm block mb-1 text-slate-700 dark:text-slate-300">
              Select New File (Optional)
            </label>
```

```
            <input
              type="file"
              onChange={(e) => setUpdateFile(e.target.files?.[0] || null)}
              className="w-full text-sm text-slate-700 dark:text-slate-300 file:mr-3 file:rounded-m
            />
          </div>

          {/* URL input */}
          <div>
            <label className="text-sm block mb-1 text-slate-700 dark:text-slate-300">
              New External URL (Optional)
            </label>
            <input
              type="url"
              value={updateUrl}
              onChange={(e) => setUpdateUrl(e.target.value)}
              placeholder="https://example.com"
              className="w-full rounded-lg border border-slate-300 dark:border-slate-600 bg-white d
            />
            {currentUrl && (
              <p className="mt-1 text-xs text-slate-500 dark:text-slate-400 break-all">
                Current URL: {currentUrl}
              </p>
            )}
          </div>

          <p className="text-xs text-slate-500 dark:text-slate-400">
            Name, description, client/program assignments will remain unchanged.
            Only file content or URL will be replaced.
          </p>

          {/* Buttons */}
          <div className="flex justify-end gap-3 pt-2">
            <button
              onClick={onClose}
```

```
            disabled={loading}
            className="px-4 py-2 rounded-lg border border-slate-300 dark:border-slate-600 text-sm
          >
            Cancel
          </button>

          <button
            onClick={onConfirm}
            disabled={loading}
            className="px-4 py-2 rounded-lg bg-blue-600 hover:bg-blue-700 text-white text-sm flex
          >
            {loading && <Loader2 className="h-4 w-4 animate-spin" />}
            Update
          </button>
        </div>
      </div>
    </div>
  );
}
```

✅ STEP 1 RESULT

After this step:

- New reusable modal component is ready
- It only handles UI
- It does **not** know backend logic
- Good separation of concerns

STEP 2) Import the new modal in `UploadedFilesList.tsx`

At the top of your `UploadedFilesList.tsx`, add this import:

TypeScript

So your imports section will become like this:

```typescript
"use client";

import { useEffect, useState, useRef } from "react";
import { MoreVertical, FolderOpen, CheckCircle2 } from "lucide-react";
import { useToast, ToastContainer } from "@/components/Toast";
import { ConfirmDialog } from "@/components/ConfirmDialog";
import DescriptionDisplay from "@/components/DescriptionDisplay";
import UrlVideoPopup from "@/components/UrlVideoPopup";
import UpdateAttachmentDialog from "@/components/UpdateAttachmentDialog";
```

✅ STEP 2 RESULT

Now `UploadedFilesList.tsx` can render the update modal.

STEP 3) Add new state variables in `UploadedFilesList.tsx`

Inside your component, near your existing `useState` calls, add these **4 new states**.

Find this section:

```
const [files, setFiles] = useState<Attachment[]>([]);
const [openMenu, setOpenMenu] = useState<number | null>(null);
const [search, setSearch] = useState("");
const [fileIdToDelete, setFileIdToDelete] = useState<number | null>(null);
const [videoPopup, setVideoPopup] = useState<{
  title: string;
  videoUrl: string;
  embedUrl: string | null;
  isDirectVideo: boolean;
} | null>(null);
```

Add below it:

```
const [fileToUpdate, setFileToUpdate] = useState<Attachment | null>(null);
const [updateFile, setUpdateFile] = useState<File | null>(null);
const [updateUrl, setUpdateUrl] = useState("");
```

✅ STEP 3 RESULT

These states will control:

- `fileToUpdate` → which attachment is being updated

- `updateFile` → selected replacement file
- `updateUrl` → replacement URL
- `updateLoading` → loading spinner while update API runs

STEP 4) Add file validation helpers in `UploadedFilesList.tsx`

Since your `AttachmentUploader.tsx` already validates file type/size, we will reuse same logic here.

Add these constants and function inside `UploadedFilesList.tsx`

You can place them **below**:

TypeScript

Paste this:

```typescript
/** 1GB limit */
const MAX_FILE_SIZE = 1024 * 1024 * 1024;

/** Allowed MIME prefixes & types */
const ALLOWED_TYPES = [
  "image/",
  "application/pdf",
```

```
  "application/msword",
  "application/vnd.ms-excel",
  "application/vnd.ms-powerpoint",
  "application/vnd.openxmlformats-officedocument",
  "text/plain",
  "video/",
  "audio/",
];

function validateFile(file: File) {
  if (file.size > MAX_FILE_SIZE) {
    return "File size exceeds 1GB limit";
  }

  const isAllowed = ALLOWED_TYPES.some((type) =>
    file.type.startsWith(type)
  );

  if (!isAllowed) {
    return "Unsupported file type";
  }

  return null;
}
```

✅ STEP 4 RESULT

Now update flow will use the **same file rules** as upload flow.

This is very important because:

- consistent validation

- no weird mismatch between upload and update
- avoids bad file types entering system

STEP 5) Add `openUpdateModal()` function

Now we need a function that opens the modal when user clicks **Update**.

Add this function inside `UploadedFilesList.tsx`

Place it somewhere near your other helper functions (for example after `notifyAttachmentsUpdated()` or after `normalizeAttachment()`).

```
function openUpdateModal(file: Attachment) {
  setOpenMenu(null);
  setFileToUpdate(file);
  setUpdateFile(null);
  setUpdateUrl(file.external_url || "");
}
```

✅ STEP 5 RESULT

When user clicks Update:

- menu closes
- modal opens

- selected file is stored
- file input is reset
- if attachment is a URL asset, current URL is prefilled

This gives a good UX.

STEP 6) Add `confirmUpdateFile()` function (frontend API call)

This is the main UI action.

⚠️ Important:
Backend does **not exist yet**, but we will still add the frontend function now.

For now it will call:

```
</> TypeScript
```

That endpoint will be built in Phase 2.

Add this full function inside `UploadedFilesList.tsx`

Place it near your other async functions like `assignFileToCurrentSelection`, `unassignFile`, `confirmDeleteFile`.

```
async function confirmUpdateFile() {
  if (!fileToUpdate) return;

  if (!updateFile && !updateUrl.trim()) {
    showToast("Please select a new file or provide a new URL", "error");
    return;
  }

  // Prevent ambiguity
  if (updateFile && updateUrl.trim()) {
    showToast("Please provide either a new file or a new URL, not both", "error");
    return;
  }

  if (updateFile) {
    const validationError = validateFile(updateFile);
    if (validationError) {
      showToast(validationError, "error");
      return;
    }
  }

  setUpdateLoading(true);

  try {
    const fd = new FormData();

    if (updateFile) {
      fd.append("file", updateFile);
    }

    if (updateUrl.trim()) {
      fd.append("external_url", updateUrl.trim());
    }
```

```
      // Keep same existing values
      fd.append("name", fileToUpdate.file_name);
      fd.append("description", fileToUpdate.description || "");

      const res = await fetch(
        `${API}/api/attachments/${fileToUpdate.id}/replace/`,
        {
          method: "POST",
          body: fd,
          credentials: "include",
        }
      );

      const data = await res.json().catch(() => ({}));

      if (!res.ok) {
        throw new Error(data?.error || "Failed to update file");
      }

      const updatedAttachment = data?.attachment
        ? normalizeAttachment(data.attachment)
        : null;

      // instant UI refresh
      if (updatedAttachment) {
        setFiles((prev) =>
          prev.map((f) => (f.id === updatedAttachment.id ? updatedAttachment : f))
        );
      }

      // close modal + reset state
      setFileToUpdate(null);
      setUpdateFile(null);
      setUpdateUrl("");
```

```
      showToast("File updated successfully", "success");

      notifyAttachmentsUpdated();

      // safest final refresh
      await fetchFilesREST();
    } catch (err: any) {
      showToast(err.message || "Update failed", "error");
    } finally {
      setUpdateLoading(false);
    }
  }
```

✅ STEP 6 RESULT

This function now does:

- ensures something is provided
- blocks user from sending both file + URL at same time
- validates file if selected
- sends `FormData`
- expects backend response in same normalized attachment shape
- updates local state
- closes modal
- refreshes list

⚠️ Until backend exists, clicking Update will show **error toast** — that is normal for now.

STEP 7) Add Update button in Assigned Files menu

Now we change the card menu in the **Assigned Files** section.

Currently in your `FileCard`, inside:

```
{openMenu === file.id && (
  <div ...>
    {isAvailableSection ? (
      <>
        ...
      </>
    ) : (
      <button
        onClick={() => unassignFile(file)}
        className="w-full px-3 py-2 text-sm rounded bg-amber-500 text-white hover:bg-amber-600"
      >
        Unassign
      </button>
    )}
  </div>
)}
```

Replace ONLY the assigned section part

Replace this:

```
) : (
  <button
    onClick={() => unassignFile(file)}
    className="w-full px-3 py-2 text-sm rounded bg-amber-500 text-white hover:bg-amber-600"
  >
    Unassign
  </button>
```

TypeScript

With this:

```
) : (
  <>
    <button
      onClick={() => openUpdateModal(file)}
      className="w-full px-3 py-2 text-sm rounded bg-blue-600 text-white hover:bg-blue-700"
    >
      Update
    </button>

    <button
      onClick={() => unassignFile(file)}
      className="w-full px-3 py-2 text-sm rounded bg-amber-500 text-white hover:bg-amber-600"
    >
      Unassign
    </button>
  </>
)}
```

✅ STEP 7 RESULT

Now in **Assigned Files** dropdown menu you will see:

- **Update**
- **Unassign**

Exactly as required.

STEP 8) Render the Update modal in JSX

Now we must actually render the modal in the component tree.

In your JSX, you already have:

- `ConfirmDialog`
- `ToastContainer`
- `UrlVideoPopup`

We will add `UpdateAttachmentDialog` there.

Find this section in `return (...)`

You currently have:

```
<ConfirmDialog
  open={fileIdToDelete != null}
  title="Delete file?"
```

```
  description="This action cannot be undone."
  confirmLabel="Delete"
  cancelLabel="Cancel"
  variant="danger"
  onConfirm={confirmDeleteFile}
  onCancel={() => setFileIdToDelete(null)}
/>

<ToastContainer toasts={toasts} onClose={removeToast} />

<UrlVideoPopup
  open={videoPopup != null}
  title={videoPopup?.title || "Video Player"}
  videoUrl={videoPopup?.videoUrl || null}
  embedUrl={videoPopup?.embedUrl || null}
  isDirectVideo={videoPopup?.isDirectVideo || false}
  onClose={() => setVideoPopup(null)}
/>
```

Add this block between `ConfirmDialog` and `ToastContainer`

(or after `UrlVideoPopup`, both are fine)

```
<UpdateAttachmentDialog
  open={fileToUpdate != null}
  attachmentName={fileToUpdate?.file_name || ""}
  description={fileToUpdate?.description || ""}
  currentUrl={fileToUpdate?.external_url || null}
  updateUrl={updateUrl}
  setUpdateUrl={setUpdateUrl}
```

```
    setUpdateFile={setUpdateFile}
    onClose={() => {
      setFileToUpdate(null);
      setUpdateFile(null);
      setUpdateUrl("");
    }}
    onConfirm={confirmUpdateFile}
    loading={updateLoading}
  />
```

✅ STEP 8 RESULT

Now modal is fully connected to component state.

When user clicks **Update**:

- modal opens
- user selects file or URL
- clicking Update triggers `confirmUpdateFile()`

FRONTEND TEST CHECKLIST (VERY IMPORTANT)

Before backend, you should test **UI behavior only**.

Test 1: Assigned file menu

Go to:

- **Assigned Files**
- click 3 dots menu

Expected:

You should now see:

- **Update**
- **Unassign**

Test 2: Update modal opens

Click **Update**

Expected:

Modal opens showing:

- Asset Name (readonly)
- Description (readonly)
- New File input
- New URL input
- Cancel button
- Update button

Test 3: Cancel works

Click **Cancel**

Expected:

- modal closes
- values reset

Test 4: URL prefill for URL assets

For an attachment with `asset_type === "url"` and `external_url` exists:

Click **Update**

Expected:

- current URL should be visible / prefilled in URL field

Test 5: Validation UI

Try clicking **Update** without selecting file or entering URL

Expected:

- error toast:
  - `"Please select a new file or provide a new URL"`

Test 6: Both file + URL at same time

Select file **and** enter URL

Expected:

- error toast:

- `"Please provide either a new file or a new URL, not both"`

Test 7: API currently fails (expected until backend exists)

If you select one valid input and click Update now:

Expected:

- request goes to:
  - `/api/attachments/<id>/replace/`
- backend not found yet
- you may get:
  - `"Failed to update file"`
  - or server 404/405 depending on backend state

⚠️ **This is normal right now.**

FULL SUMMARY OF WHAT YOU HAVE TO DO NOW

1. Create new file:

- `UpdateAttachmentDialog.tsx`

2. In `UploadedFilesList.tsx`:

- add import
- add 4 new states
- add file validation constants + function
- add `openUpdateModal()`

- add `confirmUpdateFile()`
- replace assigned menu with Update + Unassign
- render `<UpdateAttachmentDialog />`

IMPORTANT NOTE ABOUT `AttachmentUploader.tsx`

For **frontend phase**, you do **NOT** need to change `AttachmentUploader.tsx`.

Your update flow is independent from upload flow.

So for now:

✅ `AttachmentUploader.tsx` → **leave as-is**
✅ Only modify `UploadedFilesList.tsx`
✅ Create `UpdateAttachmentDialog.tsx`

STRONG RECOMMENDATION BEFORE YOU EDIT

Because your file is already big, do this safely:

Before changes:

Make backup copy of `UploadedFilesList.tsx`

Example:

```
Bash
```

```bash
```

This is especially useful since you're doing many changes.

WHAT TO DO NEXT (IMPORTANT)

Your next action:

Implement **all frontend steps above** first.

Then run app and test:

```bash
```

or your normal frontend run command.

AFTER YOU FINISH UI

Send me:

"UI done, now give backend step by step"

And then I will give you **proper Django backend phase** step-by-step:

Backend Phase will include:

1. URL route
2. Django view / DRF endpoint

3. replace file or replace URL logic
4. keep assignments unchanged
5. serializer response
6. SharePoint handling (safe approach)
7. exact paste-ready code in your style

OPTIONAL (BEST HELP FOR YOU)

If you want, in the **next message** I can give you:

full ready-to-paste updated `UploadedFilesList.tsx`

(with all frontend changes already merged into your exact file)

That may be easier than manually editing many places.

If you want that, just say:

“give me full updated UploadedFilesList.tsx ready to paste”

That will save you time and reduce mistakes.

okay it is done now can you tell me what backend changes do i need to do
from rest_framework.views import APIView
from rest_framework.response import Response

```
from rest_framework import status, permissions
from django.db import transaction, IntegrityError
from django.shortcuts import get_object_or_404
from rest_framework.parsers import MultiPartParser, FormParser

from .models import SecureAttachment, SecureAttachmentAssignment
from .serializers import (
    SecureAttachmentUploadSerializer,
    SecureAttachmentListSerializer,
    SecureAttachmentUpdateSerializer,
    SecureAttachmentAssignActionSerializer,
    ProgramSerializer,
)
from .services.sharepoint import (
    upload_file_to_sharepoint,
    create_view_only_link,
    delete_file_from_sharepoint,
)
from program.models import Program
from django.http import JsonResponse
from django.middleware.csrf import get_token
from django.utils.decorators import method_decorator
from django.views.decorators.csrf import csrf_exempt


def csrf_token_view(request):
    return JsonResponse({
        "csrfToken": get_token(request)
    })
```

```
def get_attachment_queryset():
    return
SecureAttachment.objects.filter(is_active=True).prefetch_related(
        "clients",
        "programs",
        "assignments__client",
        "assignments__program",
    )


@method_decorator(csrf_exempt, name='dispatch')
class SecureAttachmentUploadAPIView(APIView):
    authentication_classes = []
    permission_classes = [permissions.AllowAny]
    parser_classes = [MultiPartParser, FormParser]

    def initial(self, request, *args, **kwargs):
        request._dont_enforce_csrf_checks = True
        super().initial(request, *args, **kwargs)

    def post(self, request):
        try:
            data = request.data.copy()
            uploaded_file = request.FILES.get("file")

            serializer = SecureAttachmentUploadSerializer(
                data=data,
```

```
            context={"request": request},
            partial=True
        )

        if not serializer.is_valid():
            return Response(serializer.errors, status=400)

        validated = serializer.validated_data

        with transaction.atomic():
            # ==============================
            # FILE UPLOAD (SharePoint)
            # ==============================
            if uploaded_file:
                existing_file = SecureAttachment.objects.filter(
                    asset_type="file",
                    file_name=uploaded_file.name,
                    is_active=True
                ).first()

                if existing_file:
                    return Response(
                        {"error": "The file you are trying to upload already
exists."},
                        status=status.HTTP_409_CONFLICT
                    )

                upload_result = upload_file_to_sharepoint(
                    file_stream=uploaded_file.file,
```

```
                filename=uploaded_file.name,
                content_type=uploaded_file.content_type,
                file_size=uploaded_file.size,
            )

            item_id = upload_result["id"]

            existing_item = SecureAttachment.objects.filter(
                sharepoint_item_id=item_id
            ).first()

            if existing_item:
                return Response(
                    {"error": "The file you are trying to upload already
exists."},
                    status=status.HTTP_409_CONFLICT
                )

            view_url = create_view_only_link(item_id)

            attachment = SecureAttachment.objects.create(
                asset_type="file",
                file_name=uploaded_file.name,
                description=validated.get("description", ""),
                sharepoint_item_id=item_id,
                sharepoint_url=view_url,
                uploaded_by=request.user if
request.user.is_authenticated else None,
            )
```

```
            # ==============================
            # URL ONLY
            # ==============================
            else:
                attachment = SecureAttachment.objects.create(
                    asset_type="url",
                    file_name=validated["name"],
                    description=validated.get("description", ""),
                    external_url=validated["external_url"],
                    uploaded_by=request.user if
request.user.is_authenticated else None,
                )

            # TEMPORARY legacy sync
            if "clients" in validated:
                attachment.clients.set(validated["clients"])

            if "programs" in validated:
                attachment.programs.set(validated["programs"])

            # NEW exact pair mapping:
            # only create if exactly one client + one program are supplied
            upload_clients = validated.get("clients", [])
            upload_programs = validated.get("programs", [])

            if len(upload_clients) == 1 and len(upload_programs) == 1:
                selected_client = upload_clients[0]
                selected_program = upload_programs[0]
```

```
                if getattr(selected_program, "client_id", None) ==
selected_client.id:
                    SecureAttachmentAssignment.objects.get_or_create(
                        attachment=attachment,
                        client=selected_client,
                        program=selected_program,
                    )

        # Re-fetch with prefetch so serializer gets fresh assignments
        attachment = get_attachment_queryset().get(pk=attachment.pk)

        return Response(
            SecureAttachmentListSerializer(attachment).data,
            status=201,
        )

    except IntegrityError:
        return Response(
            {"error": "The file you are trying to upload already exists."},
            status=status.HTTP_409_CONFLICT
        )

    except Exception as e:
        print("ERROR in upload API:", repr(e))
        return Response(
            {"error": str(e)},
            status=status.HTTP_500_INTERNAL_SERVER_ERROR
        )
```

```
class SecureAttachmentListAPIView(APIView):
    permission_classes = [permissions.AllowAny]

    def get(self, request):
        client_id = request.GET.get("client")
        program_id = request.GET.get("program")

        queryset = get_attachment_queryset()

        # TEMPORARY legacy filter support if query params are used
        if client_id and str(client_id).isdigit():
            queryset = queryset.filter(clients__id=int(client_id))

        if program_id and str(program_id).isdigit():
            queryset = queryset.filter(programs__id=int(program_id))

        queryset = queryset.distinct()
        serializer = SecureAttachmentListSerializer(queryset, many=True)
        return Response(serializer.data, status=200)


class SecureAttachmentDetailAPIView(APIView):
    permission_classes = [permissions.AllowAny]

    def patch(self, request, pk):
        attachment = get_object_or_404(SecureAttachment, pk=pk,
is_active=True)
```

```
        serializer = SecureAttachmentUpdateSerializer(data=request.data,
partial=True)
        if not serializer.is_valid():
            return Response(serializer.errors, status=400)

        validated = serializer.validated_data

        if "clients" in validated:
            attachment.clients.set(validated["clients"])

        if "programs" in validated:
            attachment.programs.set(validated["programs"])

        attachment.save()

        # Re-fetch prefetched version
        attachment = get_attachment_queryset().get(pk=attachment.pk)

        return Response(
            SecureAttachmentListSerializer(attachment).data,
            status=200,
        )

    def delete(self, request, pk):
        attachment = get_object_or_404(SecureAttachment, pk=pk)

        try:
            with transaction.atomic():
```

```
            # If it's a SharePoint file, delete from SharePoint first
            if attachment.asset_type == "file" and
attachment.sharepoint_item_id:

delete_file_from_sharepoint(attachment.sharepoint_item_id)

            # Hard delete from DB
            attachment.delete()

        return Response({"status": "deleted"}, status=200)

    except Exception as e:
        return Response(
            {"error": f"Delete failed: {str(e)}"},
            status=status.HTTP_500_INTERNAL_SERVER_ERROR
        )


class SecureAttachmentAssignAPIView(APIView):
    permission_classes = [permissions.AllowAny]

    def post(self, request, pk):
        attachment = get_object_or_404(SecureAttachment, pk=pk,
is_active=True)

        serializer =
SecureAttachmentAssignActionSerializer(data=request.data)
        if not serializer.is_valid():
            return Response(serializer.errors, status=400)
```

```
        validated = serializer.validated_data
        client = validated["client"]
        program = validated["program"]

        # Create exact pair assignment
        assignment, created =
SecureAttachmentAssignment.objects.get_or_create(
            attachment=attachment,
            client=client,
            program=program,
        )

        # TEMPORARY: keep legacy M2M fields in sync for display
compatibility
        attachment.clients.add(client)
        attachment.programs.add(program)

        # Re-fetch prefetched version
        attachment = get_attachment_queryset().get(pk=attachment.pk)

        return Response(
            {
                "status": "assigned",
                "created": created,
                "attachment":
SecureAttachmentListSerializer(attachment).data,
            },
            status=200,
```

```
        )


class SecureAttachmentUnassignAPIView(APIView):
    permission_classes = [permissions.AllowAny]

    def post(self, request, pk):
        attachment = get_object_or_404(SecureAttachment, pk=pk,
is_active=True)

        serializer =
SecureAttachmentAssignActionSerializer(data=request.data)
        if not serializer.is_valid():
            return Response(serializer.errors, status=400)

        validated = serializer.validated_data
        client = validated["client"]
        program = validated["program"]

        deleted_count, _ = SecureAttachmentAssignment.objects.filter(
            attachment=attachment,
            client=client,
            program=program,
        ).delete()

        # TEMPORARY: rebuild legacy M2M fields from remaining exact
assignments
        remaining_assignments =
SecureAttachmentAssignment.objects.filter(
```

```
            attachment=attachment
        )

        remaining_client_ids = list(
            remaining_assignments.values_list("client_id",
flat=True).distinct()
        )
        remaining_program_ids = list(
            remaining_assignments.values_list("program_id",
flat=True).distinct()
        )

        attachment.clients.set(remaining_client_ids)
        attachment.programs.set(remaining_program_ids)

        # Re-fetch prefetched version
        attachment = get_attachment_queryset().get(pk=attachment.pk)

        return Response(
            {
                "status": "unassigned",
                "deleted": deleted_count > 0,
                "attachment":
SecureAttachmentListSerializer(attachment).data,
            },
            status=200,
        )
```

```
class ProgramsByClientAPIView(APIView):
    permission_classes = [permissions.AllowAny]

    def get(self, request):
        client_id = request.GET.get("client")

        if not client_id:
            return Response([], status=200)

        programs = Program.objects.filter(client_id=client_id)
        serializer = ProgramSerializer(programs, many=True)
        return Response(serializer.data)
```

views.py

```
from django.urls import path
from .views import (
    SecureAttachmentUploadAPIView,
    SecureAttachmentListAPIView,
    SecureAttachmentDetailAPIView,
    SecureAttachmentAssignAPIView,
    SecureAttachmentUnassignAPIView,
    ProgramsByClientAPIView,
)
from .views_lookup import (
    ClientListAPIView,
    ProgramListAPIView,
)
from graphene_django.views import GraphQLView
from django.views.decorators.csrf import csrf_exempt
from .graphql.schema import schema
```

```
from attachments.views import csrf_token_view

urlpatterns = [
    path("upload/", SecureAttachmentUploadAPIView.as_view(),
name="secure-attachment-upload"),
    path("", SecureAttachmentListAPIView.as_view(), name="secure-
attachment-list"),
    path("clients/", ClientListAPIView.as_view()),
    path("programs/", ProgramListAPIView.as_view()),
    path("programs-by-client/", ProgramsByClientAPIView.as_view()),
    path("<int:pk>/assign/", SecureAttachmentAssignAPIView.as_view(),
name="attachment-assign"),
    path("<int:pk>/unassign/",
SecureAttachmentUnassignAPIView.as_view(), name="attachment-
unassign"),
    path("<int:pk>/", SecureAttachmentDetailAPIView.as_view(),
name="attachment-detail"),
    path("graphql/", csrf_exempt(GraphQLView.as_view(schema=schema,
graphiql=True))),
    path("csrf/", csrf_token_view),
]
urls.py
from django.db import models
from django.conf import settings


class SecureAttachment(models.Model):

    ASSET_TYPE_CHOICES = (
```

```
        ("file", "File"),
        ("url", "URL"),
    )

    asset_type = models.CharField(
        max_length=50,
        choices=ASSET_TYPE_CHOICES,
        default="file"
    )

    file_name = models.CharField(max_length=255)
    description = models.TextField(blank=True)

    # SharePoint fields (only for file)
    sharepoint_item_id = models.CharField(
        max_length=255,
        unique=True,
        null=True,
        blank=True,
    )
    sharepoint_url = models.URLField(null=True, blank=True)

    # URL-only field
    external_url = models.URLField(max_length=2000, null=True,
blank=True)

    uploaded_by = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
```

```
        null=True,
        blank=True,
        related_name="uploaded_attachments"
    )

    created_at = models.DateTimeField(auto_now_add=True)

    clients = models.ManyToManyField(
        "client.Client",
        blank=True,
        related_name="attachments"
    )

    programs = models.ManyToManyField(
        "program.Program",
        blank=True,
        related_name="attachments"
    )

    sessions = models.ManyToManyField(
        "program.Session",
        blank=True,
        related_name="attachments"
    )

    is_active = models.BooleanField(default=True)

    class Meta:
        ordering = ["-created_at"]
```

```
    def __str__(self):
        return self.file_name

class SecureAttachmentAssignment(models.Model):
    attachment = models.ForeignKey(
        "attachments.SecureAttachment",
        on_delete=models.CASCADE,
        related_name="assignments"
    )
    client = models.ForeignKey(
        "client.Client",
        on_delete=models.CASCADE,
        related_name="secure_attachment_assignments"
    )
    program = models.ForeignKey(
        "program.Program",
        on_delete=models.CASCADE,
        related_name="secure_attachment_assignments"
    )
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        unique_together = ("attachment", "client", "program")
        ordering = ["-created_at"]

    def __str__(self):
        return f"{self.attachment.file_name} -> {self.client.name} /
{self.program.name}"
```

```
models.py
from django.apps import AppConfig


class AttachmentsConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'attachments'
apps.py
from django.contrib import admin
from django import forms

from .models import SecureAttachment
from .services.sharepoint import (
    upload_file_to_sharepoint,
    create_view_only_link,
)


# ----------------------------
# Admin Form
# ----------------------------
class SecureAttachmentAdminForm(forms.ModelForm):
    # 🔑 Upload field (ONLY for creation)
    upload_file = forms.FileField(required=True)

    class Meta:
        model = SecureAttachment
        fields = (
            "upload_file",
```

```
            "file_name",
            "description",
            "clients",
            "programs",
            "sessions",
        )


# ----------------------------
# Admin Configuration
# ----------------------------
@admin.register(SecureAttachment)
class SecureAttachmentAdmin(admin.ModelAdmin):
    form = SecureAttachmentAdminForm

    list_display = (
        "file_name",
        "uploaded_by",
        "created_at",
    )

    readonly_fields = (
        "sharepoint_url",
        "sharepoint_item_id",
        "created_at",
    )

    filter_horizontal = (
        "clients",
```

```
        "programs",
        "sessions",
    )

    # ----------------------------
    # Hide upload_file on edit
    # ----------------------------
    def get_form(self, request, obj=None, **kwargs):
        form = super().get_form(request, obj, **kwargs)
        if obj:  # Editing existing attachment
            form.base_fields.pop("upload_file", None)
        return form

    # ----------------------------
    # Save logic
    # ----------------------------
    def save_model(self, request, obj, form, change):
        """
        Uploads file to SharePoint ONLY on create.
        On edit, only metadata (clients/programs/sessions) is updated.
        """

        # If editing, DO NOT re-upload file
        if change:
            super().save_model(request, obj, form, change)
            return

        # ----------------------------
        # Create flow (new attachment)
```

```
        # ----------------------------
        uploaded_file = form.cleaned_data["upload_file"]

        uploaded = upload_file_to_sharepoint(
            file_stream=uploaded_file.file,
            filename=uploaded_file.name,
            content_type=uploaded_file.content_type,
            file_size=uploaded_file.size,
        )

        item_id = uploaded["id"]
        secure_link = create_view_only_link(item_id)

        obj.file_name = uploaded_file.name
        obj.sharepoint_item_id = item_id
        obj.sharepoint_url = secure_link
        obj.uploaded_by = request.user

        super().save_model(request, obj, form, change)
```

admin.py

```
import msal
import requests
from django.conf import settings
from urllib.parse import quote


# AUTH
def get_access_token():
    print("DEBUG: Requesting SharePoint access token")
```

```
    app = msal.ConfidentialClientApplication(
        client_id=settings.SHAREPOINT_CLIENT_ID,
        client_credential=settings.SHAREPOINT_CLIENT_SECRET,

authority=f"https://login.microsoftonline.com/{settings.SHAREPOINT_T
ENANT_ID}",
    )

    token = app.acquire_token_for_client(
        scopes=["https://graph.microsoft.com/.default"]
    )

    if "access_token" not in token:
        print("ERROR: Failed to get access token:", token)
        raise Exception(f"Failed to get access token: {token}")

    print("DEBUG: Access token acquired")
    return token["access_token"]


# MAIN UPLOAD ENTRY POINT
def upload_file_to_sharepoint(file_stream, filename, content_type,
file_size):
    print(
        f"DEBUG: Upload start | file={filename} | size={file_size} | type=
{content_type}"
    )
```

```
    if file_size <= 4 * 1024 * 1024:  # 4 MB
        print("DEBUG: Using simple upload")
        return _simple_upload(file_stream, filename, content_type)

    print("DEBUG: Using upload session (chunked)")
    return _upload_with_session(file_stream, filename, file_size)


# SIMPLE UPLOAD (EXACT BEHAVIOR + PRINTS KEPT)
def _simple_upload(file_stream, filename, content_type):
    token = get_access_token()
    safe_filename = quote(filename)

    url = (
        f"https://graph.microsoft.com/v1.0/sites/"
        f"{settings.SHAREPOINT_SITE_ID}"
        f"/drive/root:/LeaderboardFiles/{safe_filename}:/content"
    )

    headers = {
        "Authorization": f"Bearer {token}",
        "Content-Type": content_type,
    }

    response = requests.put(
        url,
        headers=headers,
        data=file_stream  # STREAM DIRECTLY (RAM → SharePoint)
    )
```

```
    #  ORIGINAL PRINTS (UNCHANGED)
    print("DEBUG: upload status:", response.status_code)
    print("DEBUG: upload response:", response.text)

    response.raise_for_status()
    return response.json()


# LARGE FILE UPLOAD (VIDEOS)
def _upload_with_session(file_stream, filename, file_size):
    token = get_access_token()
    safe_filename = quote(filename)

    session_url = (
        f"https://graph.microsoft.com/v1.0/sites/"
        f"{settings.SHAREPOINT_SITE_ID}"

f"/drive/root:/LeaderboardFiles/{safe_filename}:/createUploadSession"
    )

    session_response = requests.post(
        session_url,
        headers={
            "Authorization": f"Bearer {token}",
            "Content-Type": "application/json",
        },
        json={
            "item": {
```

```
            "@microsoft.graph.conflictBehavior": "rename"
        }
    },
)

print("DEBUG: upload session status:", session_response.status_code)

session_response.raise_for_status()
upload_url = session_response.json()["uploadUrl"]

print("DEBUG: upload session URL created")

CHUNK_SIZE = 10 * 1024 * 1024  # 10 MB
bytes_uploaded = 0

while True:
    chunk = file_stream.read(CHUNK_SIZE)
    if not chunk:
        break

    start = bytes_uploaded
    end = start + len(chunk) - 1

    chunk_response = requests.put(
        upload_url,
        headers={
            "Content-Length": str(len(chunk)),
            "Content-Range": f"bytes {start}-{end}/{file_size}",
        },
```

```
            data=chunk,
        )

        chunk_response.raise_for_status()
        bytes_uploaded = end + 1

    print("DEBUG: chunked upload completed")
    return chunk_response.json()


# CREATE VIEW-ONLY LINK (PRINTS KEPT + MINIMAL ADDITION)
def create_view_only_link(item_id):
    token = get_access_token()

    #  ORIGINAL PRINTS (UNCHANGED)
    print("DEBUG: Access token exists:", bool(token))
    print("DEBUG: Item ID:", item_id)

    if not token:
        raise Exception("Access token is empty")

    url = (
        f"https://graph.microsoft.com/v1.0/sites/"
        f"{settings.SHAREPOINT_SITE_ID}"
        f"/drive/items/{item_id}/createLink"
    )

    headers = {
        "Authorization": f"Bearer {token}",
```

```
        "Content-Type": "application/json",
    }

    payload = {
        "type": "view",
        "scope": "anonymous"
    }

    response = requests.post(url, headers=headers, json=payload)

    #  ORIGINAL PRINTS (UNCHANGED)
    print("DEBUG: createLink status:", response.status_code)
    print("DEBUG: createLink response:", response.text)

    response.raise_for_status()
    return response.json()["link"]["webUrl"]

# need to make scope anonymous
# DELETE FILE FROM SHAREPOINT
def delete_file_from_sharepoint(item_id):
    token = get_access_token()

    print("DEBUG: Deleting SharePoint item:", item_id)

    url = (
        f"https://graph.microsoft.com/v1.0/sites/"
        f"{settings.SHAREPOINT_SITE_ID}"
        f"/drive/items/{item_id}"
    )
```

```
    headers = {
        "Authorization": f"Bearer {token}",
    }

    response = requests.delete(url, headers=headers)

    print("DEBUG: delete status:", response.status_code)
    print("DEBUG: delete response:", response.text)

    # 204 = success
    if response.status_code not in [204, 404]:
        response.raise_for_status()

    # 404 means file already missing in SharePoint, treat as safe
    return True
```

sharepoint.py

```
from rest_framework import serializers

from .models import SecureAttachment, SecureAttachmentAssignment
from client.models import Client
from program.models import Program


#
==========================================================
=======
# BASIC LOOKUP SERIALIZERS
#
```

```
=================================================
=======
class ClientSerializer(serializers.ModelSerializer):
    class Meta:
        model = Client
        fields = ["id", "name"]


class ProgramSerializer(serializers.ModelSerializer):
    class Meta:
        model = Program
        fields = ["id", "name", "client"]


#
=================================================
=======
# UPLOAD SERIALIZER
#
=================================================
=======
class SecureAttachmentUploadSerializer(serializers.Serializer):
    name = serializers.CharField(required=False, allow_blank=True)
    description = serializers.CharField(required=False, allow_blank=True)

    external_url = serializers.URLField(
        required=False,
        allow_blank=True,
        max_length=2000
```

```
    )

    # Keep temporarily for upload compatibility
    clients = serializers.PrimaryKeyRelatedField(
        queryset=Client.objects.all(),
        many=True,
        required=False
    )

    programs = serializers.PrimaryKeyRelatedField(
        queryset=Program.objects.all(),
        many=True,
        required=False
    )

    def validate(self, data):
        file = self.context["request"].FILES.get("file")
        external_url = data.get("external_url")
        name = data.get("name")

        if not file and not external_url:
            raise serializers.ValidationError("Provide file or external_url.")

        # URL-only upload requires name
        if external_url and not file and not name:
            raise serializers.ValidationError("Name is required when
uploading external_url.")

        return data
```

```
#
==============================================
=======
# ASSIGNMENT SERIALIZERS
#
==============================================
=======
class
SecureAttachmentAssignmentSerializer(serializers.ModelSerializer):
    client = ClientSerializer(read_only=True)
    program = ProgramSerializer(read_only=True)

    class Meta:
        model = SecureAttachmentAssignment
        fields = ["id", "client", "program", "created_at"]


class SecureAttachmentAssignActionSerializer(serializers.Serializer):
    client_id = serializers.PrimaryKeyRelatedField(
        queryset=Client.objects.all(),
        source="client"
    )
    program_id = serializers.PrimaryKeyRelatedField(
        queryset=Program.objects.all(),
        source="program"
    )
```

```
    def validate(self, data):
        client = data["client"]
        program = data["program"]

        # Safety check: selected program must belong to selected client
        if getattr(program, "client_id", None) != client.id:
            raise serializers.ValidationError(
                {"program_id": "Selected program does not belong to
selected client."}
            )

        return data



#
==============================================
=======
# LIST SERIALIZER
#
==============================================
=======
class SecureAttachmentListSerializer(serializers.ModelSerializer):
    # Legacy support (temporary)
    clients = ClientSerializer(many=True, read_only=True)
    programs = ProgramSerializer(many=True, read_only=True)

    # New exact pair mapping
    assignments = SecureAttachmentAssignmentSerializer(many=True,
read_only=True)
```

```
    class Meta:
        model = SecureAttachment
        fields = (
            "id",
            "file_name",
            "description",
            "asset_type",
            "sharepoint_url",
            "external_url",
            "clients",
            "programs",
            "assignments",
            "created_at",
        )


#
==============================================
=======
# LEGACY PATCH SERIALIZER (temporary)
#
==============================================
=======
class SecureAttachmentUpdateSerializer(serializers.Serializer):
    clients = serializers.PrimaryKeyRelatedField(
        queryset=Client.objects.all(),
        many=True,
        required=False
```

```
    )
    programs = serializers.PrimaryKeyRelatedField(
        queryset=Program.objects.all(),
        many=True,
        required=False
    )
serializers.py
now tell me what backend changes to do proparlly in detail
```

Perfect — now we'll do **PHASE 2: BACKEND** properly and safely.
And yes, based on your current code, this is **very possible** and your structure is already good 👍

GOAL (Backend)

We need a new backend API endpoint:

```bash
```

This endpoint should:

- find existing `SecureAttachment`
- keep:
  - same `id`
  - same `file_name`
  - same `description`
  - same `clients`

  - same `programs`
  - same `assignments`
- only replace:
  - **SharePoint file** OR
  - **external URL**
- return updated attachment in **same serializer format** used by frontend

---

IMPORTANT DESIGN CHOICE (VERY IMPORTANT)

We will use new separate API view

Do NOT modify upload API for this.

Why?

Because:

- `upload/` = create new attachment
- `replace/` = update existing attachment content only

This keeps logic clean and prevents bugs.

EXACT BACKEND PLAN

We will do this in **7 steps**:

1. Create a **new serializer** for replace API
2. Add a **new SharePoint helper** (optional but strongly recommended)
3. Add a **new API view** `SecureAttachmentReplaceAPIView`

4. Add new URL route in `urls.py`
5. Keep old file cleanup behavior safe (recommended: optional)
6. Test with frontend
7. Verify expected response shape

STEP 1) Add a new serializer for replace API

You already have:

- `SecureAttachmentUploadSerializer`
- `SecureAttachmentUpdateSerializer`
- `SecureAttachmentAssignActionSerializer`

We should create a **separate serializer** for replace action.

Open `serializers.py`

Add this new serializer below `SecureAttachmentUpdateSerializer`

Paste this:

```
# ====================================================
# REPLACE CONTENT SERIALIZER
# ====================================================
class SecureAttachmentReplaceSerializer(serializers.Serializer):
    external_url = serializers.URLField(
```

```
        required=False,
        allow_blank=True,
        max_length=2000
    )

    def validate(self, data):
        request = self.context.get("request")
        file = request.FILES.get("file") if request else None
        external_url = (data.get("external_url") or "").strip()

        if not file and not external_url:
            raise serializers.ValidationError(
                "Provide a new file or external_url."
            )

        if file and external_url:
            raise serializers.ValidationError(
                "Provide either a new file or external_url, not both."
            )

        return data
```

✅ STEP 1 RESULT

This serializer ensures:

- file OR URL required
- both together not allowed
- this exactly matches your frontend logic

STEP 2) Import the new serializer in `views.py`

Open `views.py`

Currently you have:

```
from .serializers import (
    SecureAttachmentUploadSerializer,
    SecureAttachmentListSerializer,
    SecureAttachmentUpdateSerializer,
    SecureAttachmentAssignActionSerializer,
    ProgramSerializer,
)
```

Replace with:

```
from .serializers import (
    SecureAttachmentUploadSerializer,
    SecureAttachmentListSerializer,
    SecureAttachmentUpdateSerializer,
    SecureAttachmentAssignActionSerializer,
    SecureAttachmentReplaceSerializer,
    ProgramSerializer,
)
```

✅ STEP 2 RESULT

Now your view file can use the new replace serializer.

STEP 3) (OPTIONAL BUT STRONGLY RECOMMENDED) Add helper to safely delete old SharePoint file

This is optional, but recommended for clean code.

Important recommendation:

For first version, **do NOT delete old SharePoint file immediately** after replacement.

Why?

- safer
- easier debugging
- avoids accidental data loss if upload succeeds but something else fails later
- you can manually clean old files later

So for **version 1**, we will:

✅ Upload new file
✅ Update DB to new item ID + new URL
❌ Do NOT delete old SharePoint file automatically

This is the safest.

STEP 4) Add new API view in `views.py`

This is the main part.

We will create:

Python Run

Open `views.py`

Add this full class below `SecureAttachmentUploadAPIView`

(You can place it after upload view and before list view)

Paste this:

```python
@method_decorator(csrf_exempt, name='dispatch')
class SecureAttachmentReplaceAPIView(APIView):
    authentication_classes = []
    permission_classes = [permissions.AllowAny]
    parser_classes = [MultiPartParser, FormParser]

    def initial(self, request, *args, **kwargs):
        request._dont_enforce_csrf_checks = True
        super().initial(request, *args, **kwargs)

    def post(self, request, pk):
        attachment = get_object_or_404(SecureAttachment, pk=pk, is_active=True)
```

```
        serializer = SecureAttachmentReplaceSerializer(
            data=request.data,
            context={"request": request},
            partial=True,
        )

        if not serializer.is_valid():
            return Response(serializer.errors, status=400)

        validated = serializer.validated_data
        uploaded_file = request.FILES.get("file")
        external_url = (validated.get("external_url") or "").strip()

        try:
            with transaction.atomic():
                # Keep old values only for safety/logging if needed
                old_sharepoint_item_id = attachment.sharepoint_item_id
                old_sharepoint_url = attachment.sharepoint_url

                # ====================================
                # CASE 1: REPLACE WITH NEW FILE
                # ====================================
                if uploaded_file:
                    # Optional duplicate name check (recommended)
                    # Allow same current attachment, block others with same active file name
                    existing_file = SecureAttachment.objects.filter(
                        asset_type="file",
                        file_name=uploaded_file.name,
                        is_active=True
                    ).exclude(pk=attachment.pk).first()

                    if existing_file:
                        return Response(
                            {"error": "Another active file with this file name already exists."},
```

```
            status=status.HTTP_409_CONFLICT
        )

    upload_result = upload_file_to_sharepoint(
        file_stream=uploaded_file.file,
        filename=uploaded_file.name,
        content_type=uploaded_file.content_type,
        file_size=uploaded_file.size,
    )

    new_item_id = upload_result["id"]

    existing_item = SecureAttachment.objects.filter(
        sharepoint_item_id=new_item_id
    ).exclude(pk=attachment.pk).first()

    if existing_item:
        return Response(
            {"error": "The uploaded SharePoint item already exists in another att
            status=status.HTTP_409_CONFLICT
        )

    new_view_url = create_view_only_link(new_item_id)

    # IMPORTANT:
    # Keep SAME attachment row, SAME id, SAME assignments
    # Update only content-related fields
    attachment.asset_type = "file"

    # Requirement says keep existing name unchanged
    # So DO NOT set attachment.file_name = uploaded_file.name
    # Keep old attachment.file_name as-is

    attachment.sharepoint_item_id = new_item_id
    attachment.sharepoint_url = new_view_url
```

```
                attachment.external_url = None
                attachment.save()

                # SAFETY RECOMMENDATION:
                # For v1, do NOT delete old SharePoint file automatically.
                # If you want later, we can add delayed cleanup.
                # old_sharepoint_item_id can be used later if needed.

            # ====================================
            # CASE 2: REPLACE WITH NEW URL
            # ====================================
            elif external_url:
                attachment.asset_type = "url"
                attachment.external_url = external_url
                attachment.sharepoint_item_id = None
                attachment.sharepoint_url = None
                attachment.save()

                # SAFETY RECOMMENDATION:
                # If old attachment was SharePoint file, do NOT auto-delete for v1.
                # That prevents accidental data loss.

        # Re-fetch with prefetch so serializer gets fresh assignments
        attachment = get_attachment_queryset().get(pk=attachment.pk)

        return Response(
            {
                "status": "replaced",
                "attachment": SecureAttachmentListSerializer(attachment).data,
            },
            status=200,
        )

    except IntegrityError:
        return Response(
```

```
                {"error": "A conflicting attachment already exists."},
                status=status.HTTP_409_CONFLICT
            )

        except Exception as e:
            print("ERROR in replace API:", repr(e))
            return Response(
                {"error": str(e)},
                status=status.HTTP_500_INTERNAL_SERVER_ERROR
            )
```

✅ STEP 4 RESULT

This new endpoint now does:

If new file is sent:

- uploads new file to SharePoint
- generates new view link
- updates existing attachment row
- keeps same `id`
- keeps same `file_name`
- keeps same assignments
- keeps same client/program M2M
- changes:
  - `asset_type = "file"`
  - `sharepoint_item_id`
  - `sharepoint_url`

- `external_url = None`

If new URL is sent:

- changes:
  - `asset_type = "url"`
  - `external_url = new URL`
  - `sharepoint_item_id = None`
  - `sharepoint_url = None`

Does NOT touch:

- `file_name`
- `description`
- `clients`
- `programs`
- `assignments`

That is **exactly your requirement**.

STEP 5) Add the new route in `urls.py`

Now frontend is calling:

```
</> TypeScript
```

So backend must expose it.

Open `urls.py`

Currently imports are:

```
from .views import (
    SecureAttachmentUploadAPIView,
    SecureAttachmentListAPIView,
    SecureAttachmentDetailAPIView,
    SecureAttachmentAssignAPIView,
    SecureAttachmentUnassignAPIView,
    ProgramsByClientAPIView,
)
```

Replace with:

```
from .views import (
    SecureAttachmentUploadAPIView,
    SecureAttachmentReplaceAPIView,
    SecureAttachmentListAPIView,
    SecureAttachmentDetailAPIView,
    SecureAttachmentAssignAPIView,
    SecureAttachmentUnassignAPIView,
    ProgramsByClientAPIView,
)
```

Now add new path in `urlpatterns`

Current relevant section:

```
urlpatterns = [
    path("upload/", SecureAttachmentUploadAPIView.as_view(), name="secure-attachment-upload"),
    path("", SecureAttachmentListAPIView.as_view(), name="secure-attachment-list"),
    path("clients/", ClientListAPIView.as_view()),
    path("programs/", ProgramListAPIView.as_view()),
    path("programs-by-client/", ProgramsByClientAPIView.as_view()),
    path("<int:pk>/assign/", SecureAttachmentAssignAPIView.as_view(), name="attachment-assign"),
    path("<int:pk>/unassign/", SecureAttachmentUnassignAPIView.as_view(), name="attachment-unassi
    path("<int:pk>/", SecureAttachmentDetailAPIView.as_view(), name="attachment-detail"),
    path("graphql/", csrf_exempt(GraphQLView.as_view(schema=schema, graphiql=True))),
    path("csrf/", csrf_token_view),
]
```

Add this line after upload path (recommended position):

```
path("<int:pk>/replace/", SecureAttachmentReplaceAPIView.as_view(), name="attachment-replace"),
```

Python Run

Final relevant section becomes:

```
urlpatterns = [
    path("upload/", SecureAttachmentUploadAPIView.as_view(), name="secure-attachment-upload"),
    path("<int:pk>/replace/", SecureAttachmentReplaceAPIView.as_view(), name="attachment-replace"
    path("", SecureAttachmentListAPIView.as_view(), name="secure-attachment-list"),
    path("clients/", ClientListAPIView.as_view()),
    path("programs/", ProgramListAPIView.as_view()),
    path("programs-by-client/", ProgramsByClientAPIView.as_view()),
    path("<int:pk>/assign/", SecureAttachmentAssignAPIView.as_view(), name="attachment-assign"),
    path("<int:pk>/unassign/", SecureAttachmentUnassignAPIView.as_view(), name="attachment-unassi
    path("<int:pk>/", SecureAttachmentDetailAPIView.as_view(), name="attachment-detail"),
    path("graphql/", csrf_exempt(GraphQLView.as_view(schema=schema, graphiql=True))),
    path("csrf/", csrf_token_view),
]
```

✅ STEP 5 RESULT

Now frontend endpoint path matches backend.

STEP 6) NO model changes required ✅

Good news:

You do NOT need to change:

- `models.py`
- migrations
- admin.py

Because your existing model already supports both:

- file
- URL

And already stores:

- `sharepoint_item_id`
- `sharepoint_url`
- `external_url`

So:

✅ **No migration needed**
✅ **No model change needed**

STEP 7) NO change required in `SecureAttachmentDetailAPIView` (recommended)

You already have:

- `PATCH` for legacy M2M metadata
- `DELETE` for deleting attachment

Do **not** overload this for replace content right now.

Why?

Keeping replace separate is better:

- less risk
- clearer API
- easier frontend maintenance

- easier debugging

So:

✅ Keep `SecureAttachmentDetailAPIView` as-is

STEP 8) IMPORTANT behavior note about file name

You explicitly said:

> keep its name and assigned clients and programs as it is
> just update file content or URL

So in backend I intentionally **did NOT change**:

Python Run

Example:

If existing attachment name is:

```
Leadership Training PDF
```

and user uploads a file named:

```
new_final_version_2.pdf
```

The attachment will still show:

```
Leadership Training PDF
```

This matches your requirement.

STEP 9) IMPORTANT behavior note about description

You said:

- keep name
- keep assignments
- update only content / URL

So I also intentionally **did NOT change**:

```python
```

Even though frontend sends:

- `name`
- `description`

We ignore them in replace endpoint for now.

That is correct and safe.

STEP 10) IMPORTANT behavior note about old SharePoint file cleanup

This is the most important practical decision.

Recommended for Version 1 (what I strongly suggest)

When replacing a SharePoint file:

Do:

- upload new file
- update DB to new item ID + new view URL

Do NOT do yet:

- auto-delete old SharePoint file

Why?

Because if you auto-delete old file:

- and something goes wrong after upload
- or wrong item ID is used
- or user complains
- recovery becomes harder

For now:

✅ safer
✅ easier to debug
✅ no accidental data loss

Later (optional enhancement)

If you want later, we can add:

- after successful DB save
- try deleting old SharePoint item in a `try/except`
- ignore failure if old file already missing

But **not recommended in first version**.

STEP 11) Expected frontend response shape

Your frontend currently expects:

```typescript
const updatedAttachment = data?.attachment
  ? normalizeAttachment(data.attachment)
```

TypeScript

So backend must return:

```
{
  "status": "replaced",
  "attachment": {
    "id": 1,
    "file_name": "...",
    "description": "...",
    "asset_type": "file" or "url",
    "sharepoint_url": "...",
    "external_url": "...",
    "clients": [...],
```

```
    "programs": [...],
    "assignments": [...],
    "created_at": "..."
  }
}
```

The code I gave already returns exactly that ✅

STEP 12) Run server and test

After saving all backend changes:

Run Django server:

```bash
```

STEP 13) Test scenarios (VERY IMPORTANT)

Now test these one by one.

TEST 1: Assigned file → Update → Replace with new URL

Steps:

- open frontend
- go to assigned file

- click **Update**
- leave file empty
- enter new URL
- click **Update**

Expected:

- success toast
- file remains in **Assigned Files**
- same name remains
- same assignments remain
- if it was file before:
  - now `asset_type = "url"`
  - `sharepoint_url = null`
  - `external_url = new url`

TEST 2: Assigned file → Update → Replace with new file

Steps:

- open assigned file
- click **Update**
- choose new file
- leave URL empty
- click **Update**

Expected:

- new file uploaded to SharePoint

- same attachment row remains
- same name remains
- same assignments remain
- now:
  - `asset_type = "file"`
  - `sharepoint_item_id` updated
  - `sharepoint_url` updated
  - `external_url = null`

TEST 3: Both file + URL together

Expected:

- frontend blocks it
- backend also blocks it (double safety)

TEST 4: Neither file nor URL

Expected:

- frontend blocks it
- backend also blocks it

TEST 5: Verify assignment remains

After update:

- attachment should still appear in **Assigned Files** for same selected client/program
- it should **not move to Available Files**

This is the main correctness check.

STEP 14) Small improvement I recommend (optional but good)

Right now your replace endpoint allows `AllowAny` and `csrf_exempt`, same as upload.

That matches your current setup, so it will work.

But later for production, you should ideally move to:

- authenticated requests
- CSRF properly enabled for session auth

For now:

✅ keep consistent with your existing upload API
So no need to change now.

STEP 15) Very important note about your existing upload API behavior

In upload API, when file is uploaded, you do:

Python Run

That is fine for create.

In replace API, I intentionally did:

```python
```

This is the difference between:

- **create**
- **replace existing content**

That's exactly what you wanted.

FINAL EXACT FILE CHANGES SUMMARY

---

1) `serializers.py`

Add:

```python
class SecureAttachmentReplaceSerializer(serializers.Serializer):
```

---

2) `views.py`

Import:

Python Run

Add new class:

```python
class SecureAttachmentReplaceAPIView(APIView):
```

Python Run

---

3) `urls.py`

Import:

Python Run

Add route:

```python
path("<int:pk>/replace/", SecureAttachmentReplaceAPIView.as_view(), name="attachment-replace"),
```

Python Run

---

4) `models.py`

No changes

---

5) `admin.py`

No changes

---

6) `sharepoint.py`

No changes required for first version

MY STRONG RECOMMENDATION

Implement exactly this first

Because it is:

- safe
- simple
- minimal
- matches your frontend
- preserves assignments properly
- avoids risky SharePoint deletion logic

NEXT BEST STEP (HIGHLY RECOMMENDED)

To avoid mistakes, I can now give you:

full ready-to-paste updated `views.py`

with:

- your existing code
- new replace API inserted in correct place
- imports fixed

and then

full ready-to-paste updated `serializers.py`

and

full ready-to-paste updated `urls.py`

That will be much safer than manual editing.

Reply exactly:

“give me ready-to-paste views.py serializers.py urls.py”

and I’ll give you the **full final backend code** in your exact project style, properly merged.